

SESSION 7

# Cost, Latency & Model Choice

Prompts have a bill and a clock. Engineering them means managing both.

Mardi 29 septembre · 8h00-9h45 · 1h45

Prompt & Context Engineering — Stefan Duprey



TODAY

# Session 7 — plan & goals

## You will be able to

- Choose between merge and rebase; use stash
- Reason about token cost & latency
- Pick the right model for the task
- Apply reliability patterns

## RUN OF SHOW

|      |                                          |
|------|------------------------------------------|
| 0:00 | Git — merge vs rebase, stash             |
| 0:20 | Cost, latency & model choice             |
| 0:55 | Lab — small vs frontier: quality vs cost |
| 1:40 | Capstone check-in                        |

# Merge vs rebase

**M**

## Merge

- Keeps the true history intact
- Adds a merge commit at the join
- Safe for shared / pushed branches
- History can get tangled over time

**R**

## Rebase

- Replays your commits on a new base
- Produces a clean, linear history
- Great to tidy up before a PR
- Never rebase commits others already have

# Every prompt has a bill

Every prompt you write has a price tag and a stopwatch attached — good engineers keep both in view.

- **You pay per token, every call**

Input tokens and output tokens both cost money. A verbose prompt run millions of times is a real budget line.

- **Long context is slow as well as pricey**

Latency grows with token count. Users feel a slow assistant even when the answer is perfect.

- **A 'better' prompt can be a worse product**

If a tweak doubles cost or latency for a tiny quality gain, it may not be worth shipping.



## MINDSET

Optimise quality per token, not quality at any cost.  
Put price and latency next to the score in your eval.

# Cut cost & latency

Four practical levers to make the same prompt cheaper and faster, ideally without a quality hit.



## Shorter prompts

Trim boilerplate and redundant examples — then re-run the eval to confirm quality held. Fewer tokens, lower cost and latency.



## Caching

Reuse a cached prompt prefix or prior results for repeated context. Big savings when many calls share the same setup.



## Batching

Group requests and run them asynchronously to raise throughput when you have many to process.



## Streaming

Stream tokens to the user so they see output immediately. Same total time, far better perceived latency.

# Small vs frontier model

Bigger isn't automatically better — the real skill is matching the model to the job in front of you.

## **S** Small / local

- Cheap, fast, private, runs offline
- Great for classification & extraction
- Needs more few-shot and structure
- Struggles with hard, open-ended reasoning

## **F** Frontier / hosted

- Strong reasoning out of the box
- Best for open-ended, multi-step work
- Expensive and slower per call
- Your data leaves the machine

# Compress the context

The cheapest, fastest token is the one you never sent — here's how to trim without losing quality.

- **Summarize history, don't resend it**

In multi-turn apps, replace old turns with a running summary instead of pasting the whole transcript every call.

- **Retrieve fewer, better chunks**

Tie back to S5: tighter retrieval means less context, lower cost, and often better answers.

- **Drop examples the eval says you don't need**

Every few-shot example costs tokens on every call. Keep only the ones that actually move the score.



## CHECK

Every token you remove is cheaper and faster — then re-run the eval to confirm the quality actually held.

# Reliability patterns

The unglamorous patterns that separate a demo from something you'd actually put in front of users.

- **Retry with backoff**

Transient API failures happen. Retry sensibly instead of crashing the user's request.

- **Fall back to a bigger model only when needed**

Try the cheap model first; escalate to the expensive one only when the cheap one fails a check.

- **Validate structure; repair or reject**

Never trust raw output. Validate against the schema (S4) and handle failure explicitly.

- **Pin temperature 0 for repeatability**

When you need the same input to give the same output, remove the randomness.



## GOAL

A system that's cheap, fast and reliable beats one that's occasionally brilliant and often broken.



# Lab — Quality vs cost

LOCAL + HOSTED

`prompt-engineering-course/07_cost_latency`

## TASKS

1. Run one task on a small local model and a frontier model
2. Log tokens, latency and eval score for each
3. Trim the prompt, then re-measure all three
4. Recommend a model + prompt for this task, with the numbers



## DELIVERABLE

A small table of quality vs cost vs latency across two models, and a justified recommendation committed to the repo.

# Recap — Session 7



Prompts cost tokens, money and time



Cut cost with shorter prompts, caching, batching, streaming



Match model size to the task — small often wins



Put price and latency beside quality in your eval

## NEXT SESSION

### Safety & security

Prompt injection (direct and via documents), jailbreaks, data exfiltration, and building guardrails.